

Lecture 5

Boosting

Machine Learning I

Alexander Quispe

ENEI – INEI · PEU-CD 2026

September 21, 2026

Outline

Additive Models and Forward Stagewise Fitting

Gradient Boosting

AdaBoost

Why Boosting Generalizes

Summary

Two Ways to Combine Models

Bagging (Lecture 4)

- Many *independent* deep trees.
- Each: low bias, high variance.
- Average them. Variance $\rightarrow \rho\sigma^2$, bias unchanged.
- Parallel; order does not matter.

Boosting (today)

- Many *dependent* shallow trees.
- Each: high bias, low variance.
- Add them one at a time, each correcting the last. Bias falls; variance is controlled by the step size.
- Sequential; order is everything.

The question (Kearns & Valiant, 1988)

If a learner can always do slightly better than chance, can it be turned into one that does arbitrarily well?

Schapire (1990) proved yes. AdaBoost (Freund & Schapire, 1995) made it practical. Gradient boosting (Friedman, 2001) explained what it was doing. We take them in reverse order.

Additive Models and Forward Stagewise Fitting

Additive Models

The hypothesis class is sums of M **base learners** from a simple class $\mathcal{H}_0$ (stumps, depth-3 trees, ...):

$$f_M(\mathbf{x}) = \sum_{m=1}^M \alpha_m h_m(\mathbf{x}), \quad h_m \in \mathcal{H}_0, \alpha_m \in \mathbb{R}.$$

Fitting all (α_m, h_m) jointly is intractable for trees. Instead, **forward stagewise**:

$$(\alpha_m, h_m) = \arg \min_{\alpha, h \in \mathcal{H}_0} \sum_{i=1}^N L(y_i, f_{m-1}(\mathbf{x}_i) + \alpha h(\mathbf{x}_i)), \quad f_m = f_{m-1} + \alpha_m h_m.$$

Freeze what you have; add the one term that helps most. Never revisit.

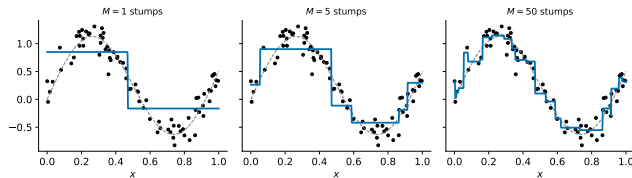
Squared Loss: Fit the Residuals

Stagewise least squares

With $L(y, \hat{f}) = \frac{1}{2}(y - \hat{f})^2$ and $r_i^{(m)} = y_i - f_{m-1}(\mathbf{x}_i)$ the current residuals,

$$\sum_i L(y_i, f_{m-1}(\mathbf{x}_i) + \alpha h(\mathbf{x}_i)) = \frac{1}{2} \sum_i (r_i^{(m)} - \alpha h(\mathbf{x}_i))^2.$$

For $\alpha = 1$ this is the least-squares fit of h to the residuals. So: *fit a base learner to whatever the current model gets wrong, add it, repeat.*



Stumps on a 1-D target. Each stage fits the residual of the previous stage; the sum approaches the curve.

Gradient Boosting

Optimization in Function Space

Consider the empirical risk as a function of the N fitted values $\mathbf{f} = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_N))^T \in \mathbb{R}^N$:

$$J(\mathbf{f}) = \sum_{i=1}^N L(y_i, f_i).$$

Gradient descent in $\mathbb{R}^N$ would set $\mathbf{f} \leftarrow \mathbf{f} - \eta \mathbf{g}$ with

$$g_i = \left. \frac{\partial L(y_i, f_i)}{\partial f_i} \right|_{f_i=f_{m-1}(\mathbf{x}_i)}.$$

- We cannot move $\mathbf{f}$ freely: the update must be a function $h \in \mathcal{H}_0$ evaluated at the $\mathbf{x}_i$.
- So project the step onto $\mathcal{H}_0$: choose h_m as the base learner **closest to** $-\mathbf{g}$, in the least-squares sense, then pick the step length by line search.

Boosting is gradient descent in the space of functions, constrained to move along directions the base learner can represent. The residuals of the previous slide are the special case $-\mathbf{g} = \mathbf{r}$ for squared loss.

Gradient Boosting (Friedman, 2001)

Input: data, loss L , base class $\mathcal{H}_0$, rounds M , shrinkage $\nu \in (0, 1]$

```
 $f_0 \leftarrow \arg \min_c \sum_i L(y_i, c);$  // a constant: the mean, the median, the log-odds  
for  $m = 1, \dots, M$  do  
     $\tilde{r}_i \leftarrow -\frac{\partial L(y_i, f)}{\partial f} \Big|_{f=f_{m-1}(\mathbf{x}_i)}, \quad i = 1, \dots, N;$  // pseudo-residuals  
     $h_m \leftarrow \arg \min_{h \in \mathcal{H}_0} \sum_i (\tilde{r}_i - h(\mathbf{x}_i))^2;$  // fit a tree to them  
     $\alpha_m \leftarrow \arg \min_{\alpha} \sum_i L(y_i, f_{m-1}(\mathbf{x}_i) + \alpha h_m(\mathbf{x}_i));$  // line search  
     $f_m \leftarrow f_{m-1} + \nu \alpha_m h_m;$   
end  
Output:  $f_M$ 
```

Loss	$L(y, f)$	Pseudo-residual $\tilde{r}_i = -\partial L / \partial f$
Squared	$\frac{1}{2}(y - f)^2$	$y_i - f(\mathbf{x}_i)$
Absolute	$ y - f $	$\text{sign}(y_i - f(\mathbf{x}_i))$
Log (binary, $y \in \{0, 1\}$)	$-yf + \log(1 + e^f)$	$y_i - \sigma(f(\mathbf{x}_i))$
Exponential ($y \in \{\pm 1\}$)	e^{-yf}	$y_i e^{-y_i f(\mathbf{x}_i)}$

The Knobs

Number of rounds M The main capacity control. Training loss decreases monotonically; validation loss eventually rises. Choose M by early stopping on a validation set.

Shrinkage ν Scales every step. Small ν (0.01–0.1) with large M beats $\nu = 1$ with small M at the same training loss — it behaves like ℓ_2 regularization on the path (Lecture 3's early-stopping remark).

Tree depth d Sets the **interaction order**: a depth- d tree can represent interactions among at most d features. Stumps ($d = 1$) give an additive model in the features. $d \in [3, 8]$ is typical.

Subsampling Fit each tree on a random fraction (≈ 0.5) of the rows: lowers variance and cost (*stochastic* gradient boosting). Column subsampling at each split does the same, as in random forests.

Unlike random forests, boosting **does** overfit in M . It is the one hyperparameter that must be tuned every time.

Second-Order Boosting (XGBoost, 2016)

Replace the gradient step by a Newton step. At round m , expand the loss to second order in the update h :

$$\sum_i L(y_i, f_{m-1}(\mathbf{x}_i) + h(\mathbf{x}_i)) \approx \sum_i \left[L(y_i, f_{m-1}(\mathbf{x}_i)) + g_i h(\mathbf{x}_i) + \frac{1}{2} k_i h(\mathbf{x}_i)^2 \right], \quad g_i = \frac{\partial L}{\partial f}, \quad k_i = \frac{\partial^2 L}{\partial f^2}.$$

Optimal leaf values for a tree with leaves $R_1, \dots, R_T$ and weights $w_1, \dots, w_T$

$h(\mathbf{x}) = w_j$ on R_j . Add a penalty $\frac{\lambda}{2} \sum_j w_j^2 + \gamma T$. Writing $G_j = \sum_{i \in R_j} g_i$, $K_j = \sum_{i \in R_j} k_i$:

$$\sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (K_j + \lambda) w_j^2 \right] + \gamma T \implies \frac{\partial}{\partial w_j} = G_j + (K_j + \lambda) w_j = 0 \implies \boxed{w_j^* = -\frac{G_j}{K_j + \lambda}}$$

Substituting back, the objective at the optimum is $-\frac{1}{2} \sum_j \frac{G_j^2}{K_j + \lambda} + \gamma T$.

The **gain** of a split $R \rightarrow R_L \cup R_R$ is therefore $\frac{1}{2} \left[\frac{G_L^2}{K_L + \lambda} + \frac{G_R^2}{K_R + \lambda} - \frac{G^2}{K + \lambda} \right] - \gamma$: an impurity function tailored to the loss, derived rather than chosen. Splits with gain < 0 are pruned.

AdaBoost

The Setting

Binary labels $y \in \{-1, +1\}$; base learners $h : \mathbb{R}^p \rightarrow \{-1, +1\}$ (stumps, typically). Final classifier

$$H(\mathbf{x}) = \text{sign}(f_M(\mathbf{x})), \quad f_M(\mathbf{x}) = \sum_{m=1}^M \alpha_m h_m(\mathbf{x}).$$

Weak learner

Given any distribution $\mathbf{w}$ over the training points, the base learner returns h with weighted error

$$\varepsilon = \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i \leq \frac{1}{2} - \gamma, \quad \gamma > 0.$$

Slightly better than a coin flip, on *every* reweighting of the data. That is all AdaBoost assumes.

AdaBoost is forward stagewise fitting with the **exponential loss** $L(y, f) = e^{-yf}$. Everything below is derived from that one fact.

Deriving AdaBoost, Part 1: What the Weights Are

At round m , with f_{m-1} fixed, the stagewise objective is

$$\sum_{i=1}^N \exp(-y_i[f_{m-1}(\mathbf{x}_i) + \alpha h(\mathbf{x}_i)]) = \sum_{i=1}^N \underbrace{e^{-y_i f_{m-1}(\mathbf{x}_i)}}_{w_i^{(m)}} e^{-\alpha y_i h(\mathbf{x}_i)}.$$

Split the sum by whether h is right or wrong

Since $y_i h(\mathbf{x}_i) \in \{+1, -1\}$,

$$\begin{aligned} \sum_i w_i^{(m)} e^{-\alpha y_i h(\mathbf{x}_i)} &= e^{-\alpha} \sum_{i: h(\mathbf{x}_i)=y_i} w_i^{(m)} + e^{\alpha} \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i^{(m)} \\ &= e^{-\alpha} \sum_i w_i^{(m)} + (e^{\alpha} - e^{-\alpha}) \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i^{(m)}. \end{aligned}$$

For any $\alpha > 0$ the first term does not depend on h and the bracket is positive. So the best h is the one with the smallest **weighted error** $\varepsilon_m = \sum_{i: h(\mathbf{x}_i) \neq y_i} w_i^{(m)} / \sum_i w_i^{(m)}$.

The weights are the exponential losses of the current model. Points the current model gets badly wrong get heavy weight; the base learner is told to focus on them.

Deriving AdaBoost, Part 2: What α Is

Normalize the weights to sum to 1. With h_m chosen, the objective in α is

$$\phi(\alpha) = (1 - \varepsilon_m)e^{-\alpha} + \varepsilon_m e^{\alpha}.$$

Minimize over α

$$\phi'(\alpha) = -(1 - \varepsilon_m)e^{-\alpha} + \varepsilon_m e^{\alpha} = 0 \quad \implies \quad e^{2\alpha} = \frac{1 - \varepsilon_m}{\varepsilon_m} \quad \implies \quad \boxed{\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}}$$

$\phi''(\alpha) > 0$, so this is the minimum.

- $\varepsilon_m < \frac{1}{2} \implies \alpha_m > 0$: a weak learner gets a positive vote. $\varepsilon_m = \frac{1}{2} \implies \alpha_m = 0$: no vote.
- $\varepsilon_m \rightarrow 0 \implies \alpha_m \rightarrow \infty$: a perfect base learner takes over — and the algorithm stops.

The weight update follows from the definition: $w_i^{(m+1)} \propto w_i^{(m)} e^{-\alpha_m y_i h_m(\mathbf{x}_i)}$, i.e. multiply misclassified points by e^{α_m} and correctly classified ones by $e^{-\alpha_m}$, then renormalize.

The Algorithm

```
 $w_i^{(1)} \leftarrow 1/N$  for all  $i$ ;  
for  $m = 1, \dots, M$  do  
     $h_m \leftarrow$  base learner trained on  $\{(\mathbf{x}_i, y_i)\}$  with weights  $\mathbf{w}^{(m)}$ ;  
     $\varepsilon_m \leftarrow \sum_i: h_m(\mathbf{x}_i) \neq y_i w_i^{(m)}$ ;  
     $\alpha_m \leftarrow \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}$ ;  
     $w_i^{(m+1)} \leftarrow w_i^{(m)} \exp(-\alpha_m y_i h_m(\mathbf{x}_i)) / Z_m$ , with  $Z_m$  such that  $\sum_i w_i^{(m+1)} = 1$ ;  
end  
Output:  $H(\mathbf{x}) = \text{sign}\left(\sum_{m=1}^M \alpha_m h_m(\mathbf{x})\right)$ 
```

No step size, no loss to specify, one hyperparameter (M). It was the first boosting algorithm that people could actually run, and for a decade the best off-the-shelf classifier available.

Numerical Example: Five Points, Three Rounds

Points on a line, $x = (1, 2, 3, 4, 5)$, labels $y = (+, +, -, -, +)$. Base learners: stumps $h(x) = \pm \text{sign}(x - t)$, $t \in \{1.5, 2.5, 3.5, 4.5\}$.

m	weights $\mathbf{w}^{(m)}$	best stump h_m	ε_m	α_m	Z_m
1	(.2, .2, .2, .2, .2)	+1 iff $x \leq 2.5$	0.200 (misses $x=5$)	$\frac{1}{2} \ln 4 = 0.693$	0.800
2	(.125, .125, .125, .125, .5)	+1 iff $x \geq 4.5$	0.250 (misses $x=1, 2$)	$\frac{1}{2} \ln 3 = 0.549$	0.866
3	(.25, .25, .083, .083, .333)	+1 iff $x \leq 2.5$	0.333 (misses $x=5$)	$\frac{1}{2} \ln 2 = 0.347$	0.943

Round 1 to round 2, in full

Misclassified $x=5$: $w_5 \leftarrow 0.2 e^{0.693} = 0.4$. The other four: $w_i \leftarrow 0.2 e^{-0.693} = 0.1$. Sum $Z_1 = 0.4 + 4(0.1) = 0.8$. Normalize: $w_5 = 0.5$, others 0.125. Check: $Z_1 = 2\sqrt{\varepsilon_1(1-\varepsilon_1)} = 2\sqrt{0.2 \cdot 0.8} = 0.8$. ✓

After three rounds: $f_3(x) = 0.693h_1 + 0.549h_2 + 0.347h_3$. Signs: $x=1, 2 \mapsto +0.491$; $x=3, 4 \mapsto -1.589$; $x=5 \mapsto -0.491$. Training error $1/5$. The bound of the next slide gives $\prod_m Z_m = 0.800 \cdot 0.866 \cdot 0.943 = 0.653$.

The Training-Error Bound (Freund & Schapire, 1997)

Theorem

The training error of AdaBoost's final classifier satisfies

$$\frac{1}{N} \sum_{i=1}^N \mathbf{1}\{H(\mathbf{x}_i) \neq y_i\} \leq \prod_{m=1}^M Z_m = \prod_{m=1}^M 2\sqrt{\varepsilon_m(1 - \varepsilon_m)}.$$

If every $\varepsilon_m \leq \frac{1}{2} - \gamma$, this is at most $e^{-2\gamma^2 M}$.

Three steps:

1. The 0/1 loss is below the exponential loss.
2. Unravelling the weight updates expresses the final exponential loss as $\prod Z_m$.
3. With the chosen α_m , each $Z_m = 2\sqrt{\varepsilon_m(1 - \varepsilon_m)} < 1$.

Proof of the Bound

Step 1: $1\{H(\mathbf{x}) \neq y\} \leq e^{-y f_M(\mathbf{x})}$

$H(\mathbf{x}) \neq y$ iff $y f_M(\mathbf{x}) \leq 0$ iff $e^{-y f_M(\mathbf{x})} \geq 1$. When it holds the right side is ≥ 1 ; otherwise the left is 0. $\square$

Step 2: unravel the weights

By the update rule, $w_i^{(m+1)} = w_i^{(m)} e^{-\alpha_m y_i h_m(\mathbf{x}_i)} / Z_m$. Iterating from $w_i^{(1)} = 1/N$:

$$w_i^{(M+1)} = \frac{1}{N} \cdot \frac{\prod_{m=1}^M e^{-\alpha_m y_i h_m(\mathbf{x}_i)}}{\prod_{m=1}^M Z_m} = \frac{e^{-y_i \sum_m \alpha_m h_m(\mathbf{x}_i)}}{N \prod_m Z_m} = \frac{e^{-y_i f_M(\mathbf{x}_i)}}{N \prod_m Z_m}.$$

Sum over i ; the left side is a distribution, so it sums to 1: $\frac{1}{N} \sum_i e^{-y_i f_M(\mathbf{x}_i)} = \prod_{m=1}^M Z_m$. Combine with Step 1. $\square$

Step 3: compute Z_m

$$Z_m = \sum_i w_i^{(m)} e^{-\alpha_m y_i h_m(\mathbf{x}_i)} = (1 - \varepsilon_m) e^{-\alpha_m} + \varepsilon_m e^{\alpha_m} = (1 - \varepsilon_m) \sqrt{\frac{\varepsilon_m}{1 - \varepsilon_m}} + \varepsilon_m \sqrt{\frac{1 - \varepsilon_m}{\varepsilon_m}} = 2 \sqrt{\varepsilon_m (1 - \varepsilon_m)}.$$

With $\varepsilon_m = \frac{1}{2} - \gamma_m$: $Z_m = \sqrt{1 - 4\gamma_m^2} \leq e^{-2\gamma_m^2}$, since $1 - u \leq e^{-u}$. $\square$

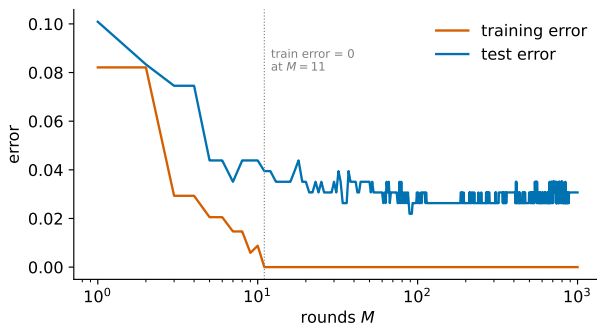
Reading the Bound

- Training error decays **exponentially** in the number of rounds, as long as the base learner stays a weak learner ($\gamma_m > 0$) on every reweighting. This answers Kearns and Valiant.
- The bound is on the exponential loss, which upper-bounds the 0/1 loss loosely: in the example, 0.653 against an actual 0.2. The mechanism is what matters, not the constant.
- The bound is on *training* error. It says nothing yet about test error — and for a while the empirical facts about test error were a puzzle.

α_m was derived assuming $\varepsilon_m < \frac{1}{2}$. If the base learner is worse than chance, flip its sign. If it is exactly chance, $\alpha_m = 0$ and the round is wasted — stop.

Why Boosting Generalizes

The Puzzle



AdaBoost with stumps on a benchmark. Training error reaches 0 early. **Test error keeps falling for hundreds of rounds after that.** Adding capacity to a model with zero training error should overfit. It does not, or not for a long time. Why?

Margins

Normalize the vote: for $y \in \{\pm 1\}$ define the **margin** of example i as

$$\mu_i = \frac{y_i \sum_m \alpha_m h_m(\mathbf{x}_i)}{\sum_m \alpha_m} \in [-1, 1].$$

$\mu_i > 0$ iff correctly classified; μ_i near 1 means a large *weighted majority* of base learners agree.

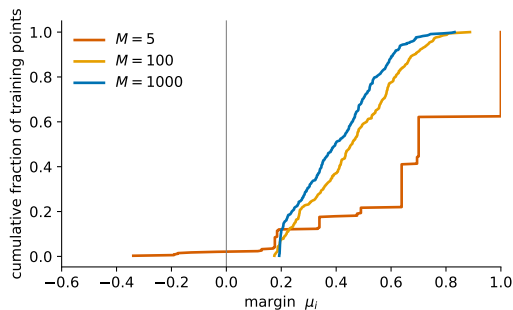
Schapire, Freund, Bartlett & Lee (1998)

With probability at least $1 - \delta$ over the training sample, for every $\theta > 0$,

$$\mathbb{P}_{\text{test}}[\mu \leq 0] \leq \underbrace{\frac{1}{N} \sum_i \mathbf{1}\{\mu_i \leq \theta\}}_{\text{fraction with margin} \leq \theta} + O\left(\sqrt{\frac{d \log^2(N/d)}{N\theta^2}} + \frac{\log(1/\delta)}{N}\right),$$

where d is the VC dimension of $\mathcal{H}_0$. **The bound does not depend on M .**

Boosting Pushes Margins Up



Cumulative distribution of training margins after 5, 100 and 1000 rounds. Training error is zero from round ≈ 20 on; what keeps changing is the *distribution*: the mass at small margins moves right. The bound on the previous slide falls with it, at fixed M -independent cost.

Why Margins Grow: The Exponential Loss

AdaBoost minimizes $\frac{1}{N} \sum_i e^{-y_i f(\mathbf{x}_i)}$. Once all $y_i f(\mathbf{x}_i) > 0$, this loss is still positive and still decreasing in each $y_i f(\mathbf{x}_i)$. The algorithm keeps going, and every round puts weight on the examples with the *smallest* $y_i f(\mathbf{x}_i)$ — exactly the ones with the smallest margin.

- Hinge loss would stop at margin 1; exponential loss never stops. That is why boosting keeps improving test error after training error is zero, and also why it eventually overfits on noisy data (it will chase mislabelled points forever).
- Log loss (LogitBoost, gradient boosting with binomial deviance) is less aggressive on outliers and is the modern default.
- The parallel with SVMs is exact: both maximize a margin. The SVM maximizes the ℓ_2 margin of one linear classifier; boosting approximately maximizes the ℓ_1 margin of a vote.

Summary

What We Proved Today

1. Forward stagewise fitting with squared loss fits each base learner to the current residuals.
2. Gradient boosting is steepest descent in function space, projected onto the base class: fit h_m to the pseudo-residuals $-\partial L / \partial f$, then line-search the step.
3. XGBoost's leaf values $w_j^* = -G_j / (K_j + \lambda)$ and its split gain follow from a second-order expansion.
4. Forward stagewise fitting with exponential loss is AdaBoost: the weights are $e^{-y_i f_{m-1}(\mathbf{x}_i)}$, the base learner minimizes weighted error, and $\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}$.
5. Training error $\leq \prod_m 2\sqrt{\varepsilon_m(1 - \varepsilon_m)} \leq e^{-2\gamma^2 M}$.
6. Test error is governed by the margin distribution, not by M ; exponential loss keeps increasing margins.

Next

Lab 3 (Wednesday, Rodrigo) — information gain by hand, trees vs. forests vs. gradient boosting on one dataset, early stopping on M , and the AdaBoost trace of this lecture reproduced in code.

Lecture 6 (Monday) — no labels at all: k -means as coordinate descent, agglomerative clustering, and principal components as the eigenvectors of the covariance — the bridge to Machine Learning II.

Reading — *ESL* §10.1–10.6 (AdaBoost, exponential loss), §10.9–10.12 (gradient boosting, shrinkage, interaction depth). Schapire et al. (1998) for margins; Chen & Guestrin (2016) for XGBoost.